

MobileDeepfake

A Cascaded Detection System for On-Device Deepfake Detection

Wing Yam Ho

The Hong Kong Polytechnic University

December 2025

Research Background

Deepfake Technology Evolution

- GAN and Diffusion-based face synthesis
- Photorealistic quality that often fools humans
- Low barrier, wide accessibility

Social Risks

- Political misinformation
- Financial fraud via celebrity/synthetic identities
- Non-consensual intimate imagery

Mobile Detection Demand

- **60%+** internet traffic from mobile
- Users need local content verification
- Privacy: sensitive data stays on-device

Key Insight

Real-time, accurate deepfake detection on mobile devices is an urgent research direction.

Detection Gaps and Mobile Challenges

Benchmark vs. Real World

- Models trained on curated datasets (FF++, CelebDF)
- **20–30% AUC drop** on in-the-wild data
- Dataset bias and domain shift issues

Server-Centric Assumptions

- Most work evaluates on server-class GPUs
- Limited attention to mobile/edge devices
- Mobile-specific metrics rarely reported

Mobile Deployment Constraints

Constraint	Target
Model Size	≤ 100 MB
Latency	< 200 ms
Energy	Battery-friendly
Privacy	On-device only

Research Gap

No existing cascade detector provides explicit FNR control with auditable cost-risk trade-offs for mobile.

Problems and Challenges

Challenge	Description
Lightweight vs. Accuracy	High-precision models are computationally heavy; mobile resources are limited
Distribution Shift	Training data differs significantly from real-world application scenarios
FNR Control	Strict false negative control under operational compute budgets
Engineering Deployment	Complete pipeline needed: from research model to deployable system

Research Questions

- How to achieve high recall under distribution shift on resource-constrained mobile

Technical Challenges in Detail

False Negative Control

- Missed fakes have **high cost** (misinformation spread)
- Must maintain $FNR < 1\%$ under compute budget
- Requires explicit, auditable threshold tuning

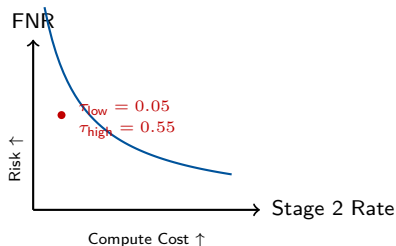
Probability Calibration

- Raw model scores often miscalibrated
- Temperature scaling for reliable confidence
- Essential for threshold-based decisions

Robustness Under Perturbations

- JPEG compression, noise, blur in real media

Trade-off Analysis



Core Insight

Explicit thresholds enable **auditable** cost-risk trade-offs, unlike black-box ensemble methods.

Research Objectives and Contributions

Four Major Contributions:

1 Cost-Aware Two-Stage Cascade

- MobileNetV4 fast filter + EfficientNetV2-B3 expert
- Dual-threshold mechanism for explicit FNR control

2 Multi-Dataset Training & Cross-Dataset Evaluation

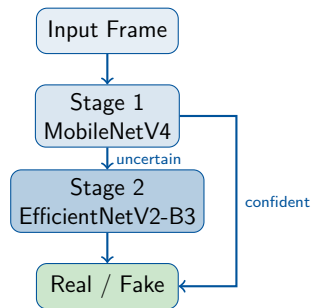
- 4 academic datasets joint training
- Deepfake-Eval-2024 independent OOD test

3 End-to-End Reproducible Pipeline

- 6-stage complete workflow

4 Mobile Deployment Solution

- INT8 quantization, ONNX export
- Android application integration



Contributions –Technical Details

1. Cost-Aware Cascade

- Stage-1 leakage + Stage-2 escalation as **system metrics**
- Cost-sensitive grid search for threshold tuning
- Optional Stage-3 LightGBM (analysis only)

2. Cross-Dataset Evaluation

- Quantify **cross-dataset gaps** and failure modes
- Compare single-model vs cascade
- Calibrated vs uncalibrated thresholds

3. Scriptable Pipeline

- Data preprocessing + manifest generation
- Hard-example mining (HEM)
- Calibration + robustness sweeps
- Scripts reproduce all thesis tables/figures

4. Mobile Export Template

- INT8 quantization + knowledge distillation
- TorchScript (primary) + ONNX export
- Metadata file: thresholds + temperature

Two-Stage Cascade Design

Design Motivation

- Target: $\sim 180\text{ms}$ latency on mobile
- Balance accuracy, energy, and speed

Stage 1: MobileNetV4

- Fast, lightweight filter
- AUC = **0.9936**
- Size: 37.3 MB

Stage 2: EfficientNetV2-B3

- Processes only uncertain samples
- AUC = 0.9633
- Size: 49.6 MB

Cascade Effect

- FNR $\approx$ **0.6%**
- Stage 2 escalation: only $\sim 1.2\%$
- Most samples resolved at Stage 1

Cascade Efficiency & Operating Points

Compute Cost Comparison

Configuration	GFLOPs	FNR	Stage2
Stage 1 only	0.54	3.12%	–
Stage 2 only	2.87	9.43%	100%
Cascade	0.59	0.60%	1.16%

Quantization Impact

- Stage 1: 37.3 MB → **10.1 MB** (INT8)
- Stage 2: 49.6 MB → **13.4 MB** (INT8)
- AUC loss: < 0.01 per stage

Deployment Scenarios

Scenario	$\tau_{\text{low}} / \tau_{\text{high}}$	FNR
Safety-first	0.05 / 0.55	0.60%
Balanced	0.10 / 0.50	1.2%
Speed-first	0.15 / 0.45	2.0%

Calibration Effect

- Temperature scaling: $T \approx 1.34$
- ECE reduced by **79%**
- Enables reliable threshold decisions

Key Insight

Cascade achieves **5× lower FNR** than Stage 1 alone with only **9%** compute

Cascade Decision Logic

Dual-Threshold Mechanism

Stage 1 output: $p_1(x) = P(\text{fake} \mid x)$

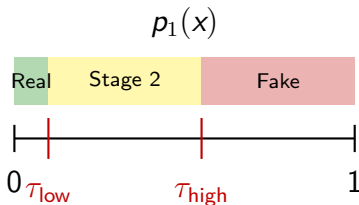
- Low threshold: $\tau_{\text{low}} = 0.05$
- High threshold: $\tau_{\text{high}} = 0.55$

Decision Rules:

$$\hat{y}(x) = \begin{cases} \text{real,} & \text{if } p_1(x) < \tau_{\text{low}} \\ \text{fake,} & \text{if } p_1(x) > \tau_{\text{high}} \\ \text{Stage2}(x), & \text{otherwise} \end{cases}$$

Stage 2 Decision:

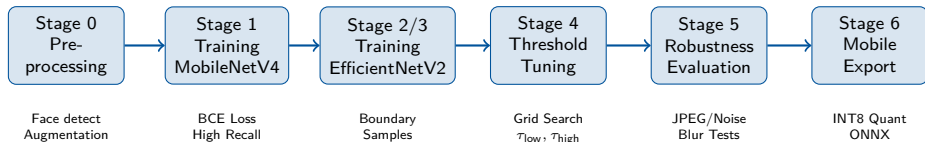
- $p_2(x) > 0.5 \Rightarrow \text{Fake}$



Performance

- FNR $\approx 0.6\%$
- Escalation rate $\approx 1.2\%$

Six-Stage Pipeline



Stage	Name	Key Techniques
0	Data & Preprocess	Multi-backend face detection, 256×256 crops, manifests
1	Stage1 Training	MobileNetV4, BCE loss, temperature calibration
2	Stage2 Training	EfficientNetV2-B3, focal loss, strong augmentation
3	Meta-Model (opt.)	LightGBM on embeddings (analysis only, not deployed)
4	Threshold Tuning	Grid search for τ_{low}/τ_{high} , FNR constraint
5	Robustness	JPEG, noise, blur, brightness, gamma sweeps
6	Mobile Export	TorchScript INT8 + FP32 ONNX, 180ms / 92.8%

Mobile Deployment Details

Export Artifacts

Format	Stage 1	Stage 2
FP32 TorchScript	37.3 MB	49.6 MB
INT8 TorchScript	10.1 MB	13.4 MB
FP32 ONNX	37.5 MB	52 MB

On-Device Pipeline

- FaceDetector API $\rightarrow$ 256×256 crop
- ImageNet normalization
- Stage 1 $\rightarrow$ cascade routing $\rightarrow$ Stage 2
- Temperature scaling on logits

On-Device Metrics (Xiaomi 13)

Metric	Value
Accuracy	92.8%
FNR	2.5%
FPR	12.5%
Stage-2 Rate	7.2%
Preprocessing	3.4 ms
Inference	176 ms
Total Latency	~ 180 ms

Runtime Stack

- ONNX Runtime Android 1.19.0
- CPUExecutionProvider (ARM NEON)
- NNAPI/GPU delegates: future work

Mobile Architecture

Model Compression

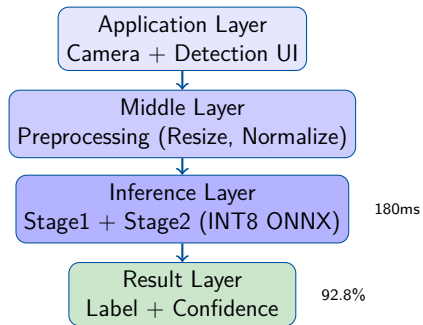
- FP32 TorchScript: 37.3MB / 49.6MB
- INT8 TorchScript: 10.1MB / 13.4MB
- FP32 ONNX (Android): ~37.5MB / ~52MB

Inference Framework

- PyTorch → ONNX export
- ONNX Runtime Android 1.19.0 (CPU)
- NNAPI/GPU delegates: future work

Key Metrics

- Latency: **~180ms**
- Accuracy: **92.8%**
- Device: Xiaomi 13 (Snapdragon 8 Gen 2)



Dataset Introduction

Training: 2.66M + OOD: 0.85M face crops

Dataset	Train	Val	Test
CelebDF-v2	83,599	17,478	18,037
FaceForensics++	223,653	50,945	47,430
DFDC	721,946	154,919	154,511
DeeperForensics	844,396	165,854	172,894
Total	1,873,594	389,196	392,872

Dataset Characteristics

- CelebDF-v2: High-quality celebrity swaps
- FF++: Multiple manipulation methods
- DFDC: Facebook large-scale diverse

OOD Evaluation: Deepfake-Eval-2024

- Val: 451,917 / Test: 402,413
- **88 websites, 52 languages**
- 2024 real media environment
- Used only for evaluation; no training

Design Goal:

- Cover diverse forgery methods
- Various shooting conditions
- Evaluate generalization capability

Dataset Preprocessing & Splits

Unified Preprocessing

- 256×256 PNG face crops
- Max 50 faces per video
- Frame interval: 10
- Multi-backend face detection

Split Protocol

- Fixed ratios: 70% / 15% / 15%
- Official test subsets preserved (FF++)
- Deterministic hash-based splitting

Balanced Manifests

- $\approx$ Equal real/fake per dataset
- Prevents dataset dominance in training

Class Distribution

Dataset	Real	Fake	Ratio
CelebDF-v2	59K	60K	0.98
FF++	161K	161K	1.00
DFDC	516K	515K	1.00
DeeperForensics	591K	592K	1.00
Eval-2024	366K	488K	0.75

Evaluation Regimes

- 1 Combined validation (all 4 datasets)
- 2 Per-dataset validation
- 3 OOD evaluation (Deepfake-Eval-2024)

Key Point

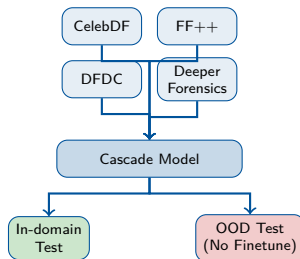
Cross-Dataset Evaluation Protocol

Training & Validation

- Only on 4 public academic datasets
- Reproducible splits (official test where available)
- Balanced manifests across all 4 datasets

Cross-Dataset Evaluation

- **No fine-tuning** on Deepfake-Eval-2024
- Direct application of trained model
- Simulates real deployment scenario



Goal

Simulate: Train on public data →
Deploy to real internet

Scenario	Metrics
----------	---------

In-domain Results

Individual Stage Performance

- Stage 1 AUC: **0.9936** (F1: 0.9561)
- Stage 2 AUC: 0.9633 (F1: 0.8930)

Cascade Performance (Safety-first)

- Cascade AUC: **0.9941**, F1: 0.9654
- Final FNR: **0.60%**
- Stage 2 Escalation: **1.16%**

Conclusion

On combined validation, cascade achieves high detection with significant compute savings.

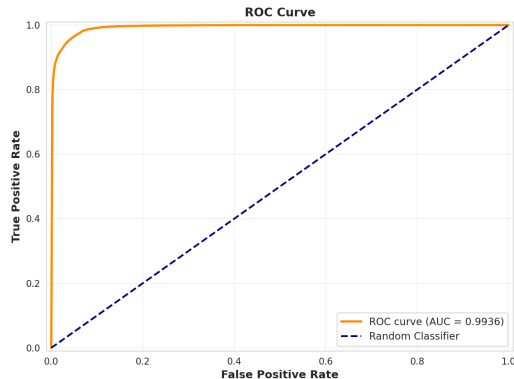


Figure: *

Stage 1 ROC Curve (AUC=0.9936)

Stage-wise Ablation Analysis

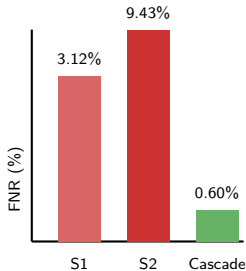
Why Cascade?

- Stage 1 alone: Fast but higher FNR
- Stage 2 alone: Lower FNR but 5 $\times$ slower
- Cascade: Best of both worlds

Stage-wise Comparison (Combined Val)

Config	AUC	F1	FNR	GFLOPs
Stage 1 only	0.9936	0.9561	3.12%	0.54
Stage 2 only	0.9633	0.8930	9.43%	2.87
Cascade	0.9941	0.9654	0.60%	0.59

FNR Reduction



Compute Efficiency

- Stage 1: 0.54 GFLOPs
- Stage 2: 2.87 GFLOPs (5.3 $\times$)
- Cascade avg: 0.59 GFLOPs

Key Insight

Cascade achieves **5 $\times$ lower FNR** than Stage 1

Cascade Efficiency Analysis

Design Philosophy

- Stage 1: Lightweight, high-throughput
- Principle: “Better catch more than miss”
- Stage 2: Fine-grained for difficult samples

Runtime Statistics (Combined Validation)

- Escalation rate: $\sim 1.2\%$
- $\Rightarrow$ **98.8%** samples only need Stage 1
- FNR maintained at $\sim 0.6\%$

Significance for Mobile/Edge

- Most samples: fast local detection
- Few samples: optional cloud refinement

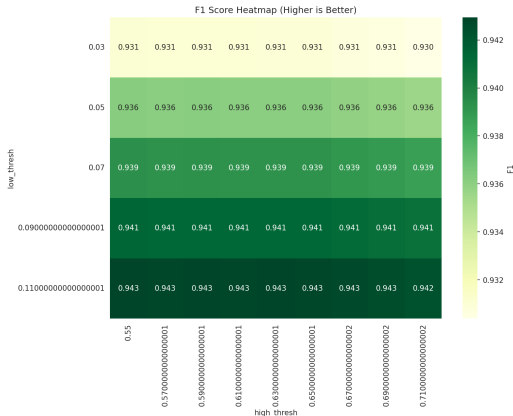


Figure: *

Mobile Performance

Test Environment

- Device: Xiaomi 13
- Chip: Snapdragon 8 Gen 2

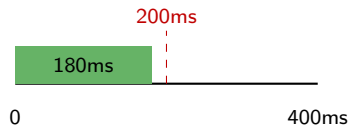
Metric	Value
Face-level Accuracy	92.8%
Average Latency	~180ms

Application Scenarios

- 25-30fps video: near real-time sampling
- On-device inference, no upload needed
- Privacy-preserving design

Deployment Feasibility

Model achieves practical real-time performance on mainstream flagship smartphones.



Cross-Dataset Challenge

Performance Drop on Deepfake-Eval-2024

Metric	In-domain	OOD
F1	>0.95	<0.30
FNR	~0.6%	73-79%
FPR	~3%	25-27%
Stage2 Rate	~1.2%	~51%

Root Causes

- Training data $\neq$ 2024 internet distribution
- New forgery methods not in training set
- Domain shift from social-media processing pipelines

Problem Revealed

Models trained only on traditional public datasets cannot directly generalize to latest real-world scenarios.

Valuable Negative Finding

This result provides an important empirical baseline for future research on cross-domain robustness.

OOD Analysis Deep Dive

Deepfake-Eval-2024 Results (Calibrated)

Metric	Val	Test
F1	0.2996	0.2796
Accuracy	58.0%	50.0%
FNR	72.9%	79.0%
FPR	26.7%	25.1%
Stage2 Rate	51.1%	52.3%

Stage2-Only Baseline (Max Compute)

Metric	Val	Test
F1	0.4460	0.5962
FNR	18.2%	14.6%
FPR	91.6%	87.1%
Stage2 Rate	97.5%	98.9%

Key Finding

Even with 100% Stage 2 usage, FPR exceeds 87% – the problem is **not just compute** but **domain coverage**.

Implications

- Cascade efficiency irrelevant if base models fail
- Need domain adaptation / continual learning
- Current training data insufficient for 2024 web

Robustness and Error Analysis

Image Perturbation Experiments

Perturbation	Observation
JPEG Compression	Moderate compression sometimes improves F1
Gaussian Noise	Moderate noise: unstable performance
Motion Blur	Near-total failure (FNR ~96-100%)

Note: Robustness results are sample-limited and treated as preliminary hypotheses.

Typical Failure Cases

- High-quality fakes: human-indistinguishable
- Extreme compression: texture destroyed
- Occlusion/non-standard: masks, filters, backlighting

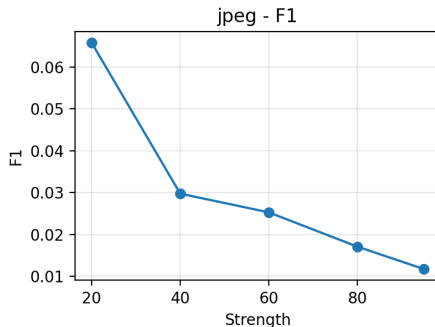


Figure: *

JPEG Quality vs F1

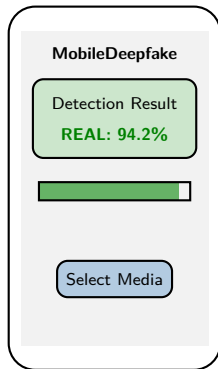
APP Interface Demo

Simple 3-Step Workflow

- 1 Import video/image (local, no cloud upload)
- 2 Local detection
- 3 Output result

Features

- Overall probability + keyframe heatmap
- Two-stage results display
- Stage 1 quick scan, Stage 2 refinement
- Average latency: $\sim 180\text{ms}$



Local Inference

Model Architecture

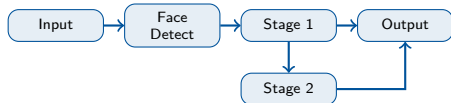
Stage	Model	FP32 TS	FP32 ONNX
Stage 1	MobileNetV4	37.3 MB	~37.5 MB
Stage 2	EfficientNetV2-B3	49.6 MB	~52 MB

Optimization & Deployment

- INT8 post-training quantization
- ONNX export for cross-platform
- ONNX Runtime Android 1.19.0
- Multi-thread pipeline (4 threads)

Engineering Highlights

- End-to-end reproducible
- Semi-automated export pipeline
- Face detection on-device
- CPU baseline (GPU/NNAPI as future work)



PC vs Mobile Deployment Comparison

Performance Comparison

Metric	PC	Mobile
Accuracy	96.5%	92.8%
FNR	0.6%	2.5%
FPR	3.5%	12.5%
Stage2 Rate	1.2%	7.2%
Latency	–	~180 ms

Model Size Comparison

Stage	FP32 TS	INT8 TS	FP32 ONNX
Stage 1	37.3 MB	10.1 MB	37.5 MB
Stage 2	49.6 MB	13.4 MB	52 MB

Android deployment uses FP32 ONNX; INT8 available for

Latency Breakdown (Xiaomi 13)

Total: ~180 ms



Inference: 176 ms

Preprocess: 3 ms

Calibration Parameters

- Temperature: $T \approx 1.34$
- ECE reduced by **79%**
- Thresholds: $\tau_{low}=0.05$, $\tau_{high}=0.55$

Work Summary

Four Major Contributions

1 Two-Stage Cascade Detection

- FNR $\approx 0.6\%$, Escalation $\approx 1.2\%$

2 Multi-Dataset Training & Evaluation

- Revealed distribution shift challenge

3 End-to-End 6-Stage Pipeline

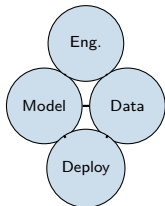
- Preprocessing to mobile export

4 Mobile Deployment

- 92.8% accuracy, $\sim 180\text{ms}$ latency

Key Metrics

Scenario	Metric	Value
PC In-domain	FNR	$\sim 0.6\%$
PC In-domain	Stage2 Rate	$\sim 1.2\%$
Mobile (Xiaomi 13)	Accuracy	92.8%
Mobile (Xiaomi 13)	Latency	$\sim 180\text{ms}$
OOD (Eval-2024)	F1	< 0.30



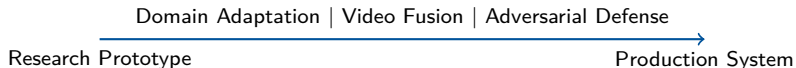
Limitations and Future Work

Current Limitations

Limitation	Description
Distribution Shift	Cross-dataset $F1 < 0.30$
Adversarial Attack	Not systematically evaluated
Device Coverage	Only Xiaomi 13 tested
Temporal Modeling	Frame-level, no video fusion

Future Directions

- **Domain Adaptation/Generalization**
 - Mitigate distribution shift
- **Temporal Modeling**
 - RNN/Transformer for video-level
- **Adversarial Defense**
 - Adversarial training
- **Multi-Platform**
 - More SoCs, iOS support



Thank You!

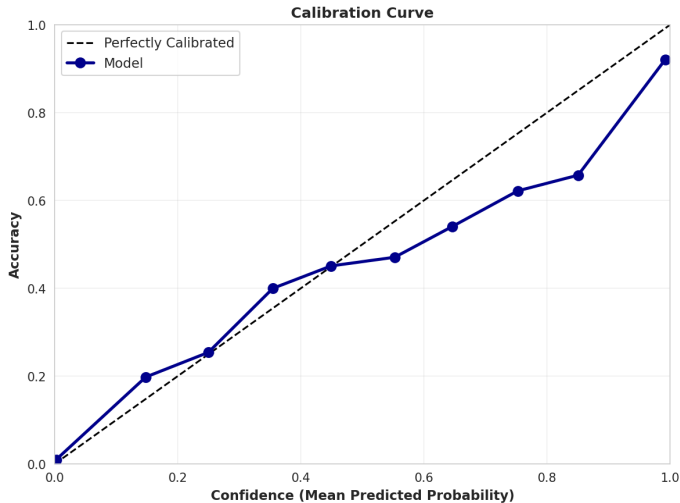
Questions & Discussion

Acknowledgments

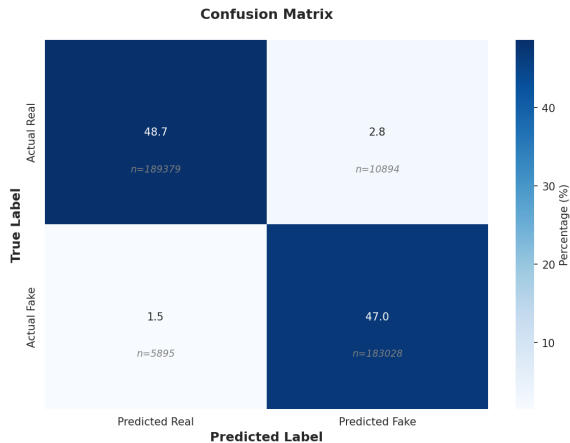
- Supervisor: [Supervisor Name]
- Research Group Members
- Open-source communities (ONNX Runtime, PyTorch)
- Public dataset contributors

This work provides a baseline platform for on-device deepfake detection research.

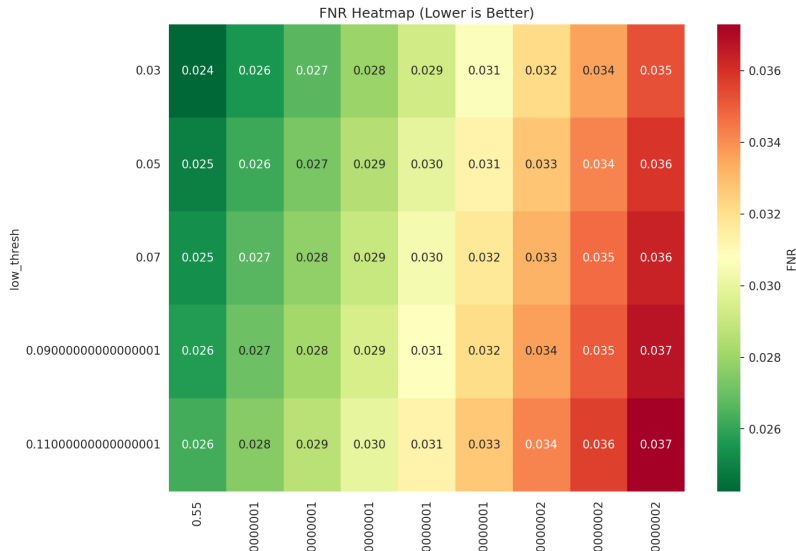
Backup: Stage1 Calibration



Backup: Stage1 Confusion Matrix



Backup: FNR Heatmap



Backup: Gaussian Noise Robustness

